

4-a)**Herança:** primeiramente foi feita uma generalização das classes **ContaCorrente** e **ContaPoupanca** para uma classe Mãe abstrata chamada **Conta**, nela está o atributo **Saldo**, que é herdado para **ContaCorrente** e **ContaPoupanca**, assim como seus métodos getter e setter. Além de seu construtor e métodos de sacar e depositar valores, é declarado um método abstrato **VerSaldo()** para ser sobrescrito tanto na **ContaCorrente** quanto **ContaPoupanca** tendo comportamentos diferentes dado o contexto. **ContaCorrente** e **ContaPoupanca** utilizam o atributo **Saldo** da classe Mãe **Conta** para os seus respectivos construtores através da palavra reservada **super**. O método **VerSaldoJuros()** de **ContaPoupanca** utiliza o atributo Saldo da classe Mãe **Conta** para seu funcionamento.

Encapsulamento: a classe Mãe **Conta** possui o atributo **Saldo privado** para uma maior segurança do código, sendo possível manipular seu atributo apenas através dos seus getters e setters pela suas subclasses. Os métodos **Sacar()** e **Depositar()** da classe Mãe **Conta** protegem o acesso direto ao saldo.

b)**Polimorfismo** em POO se refere a capacidade de um mesmo método ter diferentes comportamentos e implementações dependendo do contexto que se é aplicado. Na letra **b)** da **primeira questão** por exemplo, pede-se para que imprima o valor do saldo da classe **ContaCorrente**, chamado no código de **VerSaldo()**, então o primeiro método de **VerSaldo()** foi declarado como método abstrato na classe Mãe que não é utilizada inicialmente, mas quando se vai para as classes **ContaCorrente** e **ContaPoupanca**, **VerSaldo()** é sobrescrito para cada tipo de conta através da palavra reservada **@Override**, onde em cada classe a mesma função de ver saldo existe com comportamentos distintos a depender do tipo de conta bancária.